DATA ANALYSIS REPORT

NIK ABDUL AQMAL BIN ZUNAIDI

2026-08-21

DATA ANALYSIS MINI PROJECT: STUDENT ACADEMIC RISK

> Business Understanding: Higher Education **problem**: Predict wether a student is academically at risk or not at risk so the **Target Variable**:academic_risk (Binary Classification)**Objective**:identify at-risk students

The analysis follows the standard Data Science Process (Chapman et al., 2000), using Python libraries such as pandas (McKinney, 2010) and scikit-learn(Pedregosa et al., 2011) on the student performance dataset (Zainol, 2026)

Supporting Questions:

1. Which features (attendance, midterm, GPA) are the strongest predictors of academic risks ?
2. Does programme type influence academic risk ?
3. How does study hours per week relate to being at risk ?

PHASE 1: SETUP & LOAD THE DATA

IMPORT ALL LIBARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import(
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)
import warnings
warnings.filterwarnings('ignore')

# we need to set the style
sns.set_style('whitegrid')
RANDOM_STATE = 42

# load datasets
df = pd.read_csv('data/group01_student_performance_classification.csv')
df_raw = df.copy()
print("Shape:", df.shape)
df.head()
```

```
Shape: (12084, 14)
```

	student_id	age	gender	programme	attendance_pct	assignment_avg	quiz_avg	midte
0	ST07173	20	Male	Software Engineering	72.2	49.0	71.2	81.6
1	ST01210	25	Female	IT	75.4	66.0	76.4	56.2
2	ST11561	24	Female	Computer Science	100.0	53.6	94.6	68.5
3	ST05074	24	Male	Computer Science	72.4	100.0	65.5	78.0
4	ST05558	21	Male	Computer Science	78.7	78.5	72.4	89.0

All data manipulation is performed using pandas (McKinney, 2010), while machine learning models are built with scikit-learn (Pedregosa et al., 2011). Visualisations use matplotlib (Hunter, 2007) and seaborn (Waskom, 2021).

SAVE COPY AS ORIGINAL

```
df.to_csv('P01_data_original.csv', index=False)
print("Original dataset saved")
```

```
Original dataset saved
```

==========

PHASE 2: DATA UNDERSTANDING

BASIC STRUCTURE & MISSING VALUES

```
print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())
print("\nMissing Values (%):")
print((df.isnull().sum() / len(df) * 100).round(2))
print("\nDuplicate Rows:", df.duplicated().sum())
```

```
student_id              str
age                   int64
gender                  str
programme               str
attendance_pct      float64
assignment_avg      float64
quiz_avg            float64
midterm_score       float64
study_hours_week    float64
previous_gpa        float64
absences            float64
internet_access         str
extracurricular         str
academic_risk           str
dtype: object

Missing Values:
student_id            0
age                   0
gender                0
programme           182
attendance_pct        0
assignment_avg        0
quiz_avg              0
midterm_score       181
study_hours_week      0
previous_gpa          0
absences            181
```

```
internet_access     182
extracurricular       0
academic_risk         0
dtype: int64

Missing Values (%):
student_id          0.00
age                 0.00
gender              0.00
programme           1.51
attendance_pct      0.00
assignment_avg      0.00
quiz_avg            0.00
midterm_score       1.50
study_hours_week    0.00
previous_gpa        0.00
absences            1.50
internet_access     1.51
extracurricular     0.00
academic_risk       0.00
dtype: float64

Duplicate Rows: 83
```

TARGET DISTRIBUTION

```python
print(df['academic_risk'].value_counts())
print("\nPercentages:")
print(df['academic_risk'].value_counts(normalize=True) * 100)
# normalize true here means ?
# it means that : shows what fraction from 0 to 1 of students are each category
```

```
academic_risk
Not At Risk    10349
At Risk         1735
Name: count, dtype: int64

Percentages:
academic_risk
Not At Risk    85.642171
```

```
At Risk        14.357829
Name: proportion, dtype: float64
```

NUMERIC SUMMARY

```
numeric_cols = df.select_dtypes(include=[np.number]).columns
df[numeric_cols].describe().round(2)
```

	age	attendance_pct	assignment_avg	quiz_avg	midterm_score	study_hours_week	pre
count	12084.00	12084.00	12084.00	12084.00	11903.00	12084.00	120
mean	21.46	81.62	68.11	65.78	63.82	6.02	2.8
std	2.29	11.32	14.82	15.65	16.73	3.48	0.6
min	18.00	39.20	14.90	6.60	0.00	0.20	0.3
25%	19.00	73.90	58.30	55.20	52.70	3.50	2.4
50%	21.00	81.90	68.20	65.90	64.00	5.30	2.8
75%	23.00	90.10	78.40	76.53	75.60	7.80	3.3
max	25.00	100.00	100.00	100.00	100.00	25.00	4.0

CATEGORICAL SUMMARY

```
cat_cols = ['gender', 'programme', 'internet_access', 'extracurricular']

for col in cat_cols:
    print(f"\n-- {col} ---")
    print(df[col].value_counts(dropna=False).head(10))
```

```
-- gender ---
gender
Male      6044
Female    5920
M           64
female      56
Name: count, dtype: int64

-- programme ---
programme
```

```
Computer Science        4055
IT                      2986
Software Engineering    2829
Data Science            2032
NaN                      182
Name: count, dtype: int64

-- internet_access ---
internet_access
Yes    11234
No       668
NaN      182
Name: count, dtype: int64

-- extracurricular ---
extracurricular
No     6835
Yes    5249
Name: count, dtype: int64
```

DATA QUALITY ISSUES

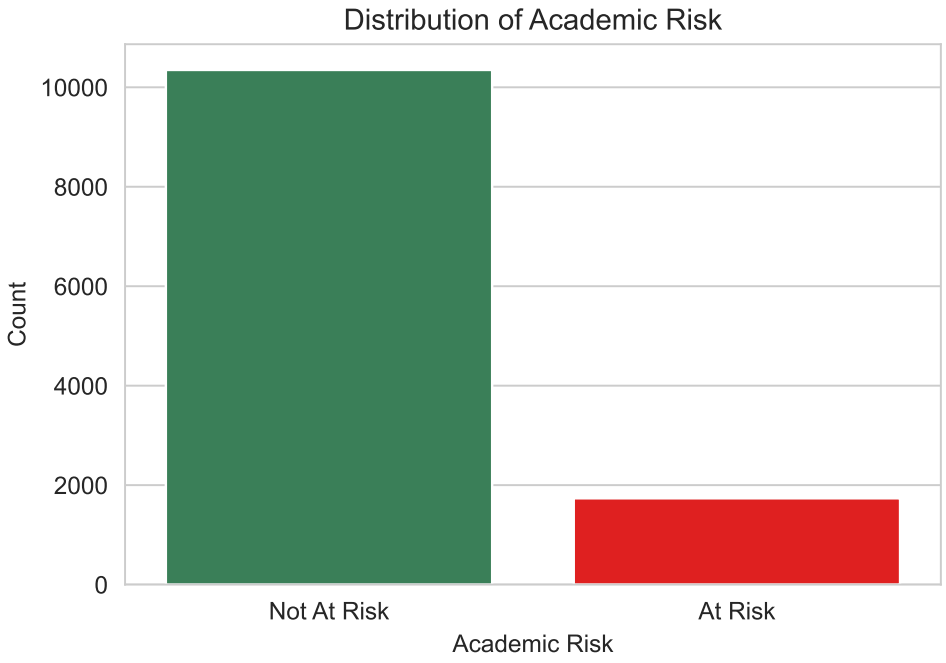
```
print("Gender unique values:", df['gender'].unique())
print("\nProgramme missing: ", df['programme'].isnull().sum())
print("Midterm missing:", df['midterm_score'].isnull().sum())
print("Absence missing:", df['absences'].isnull().sum())
print("Internet access missig", df['internet_access'].isnull().sum())
```

```
Gender unique values: <StringArray>
['Male', 'Female', 'female', 'M']
Length: 4, dtype: str

Programme missing:  182
Midterm missing: 181
Absence missing: 181
Internet access missig 182
```

VISUALIZATION

V1: TARGET DISTRIBUTION

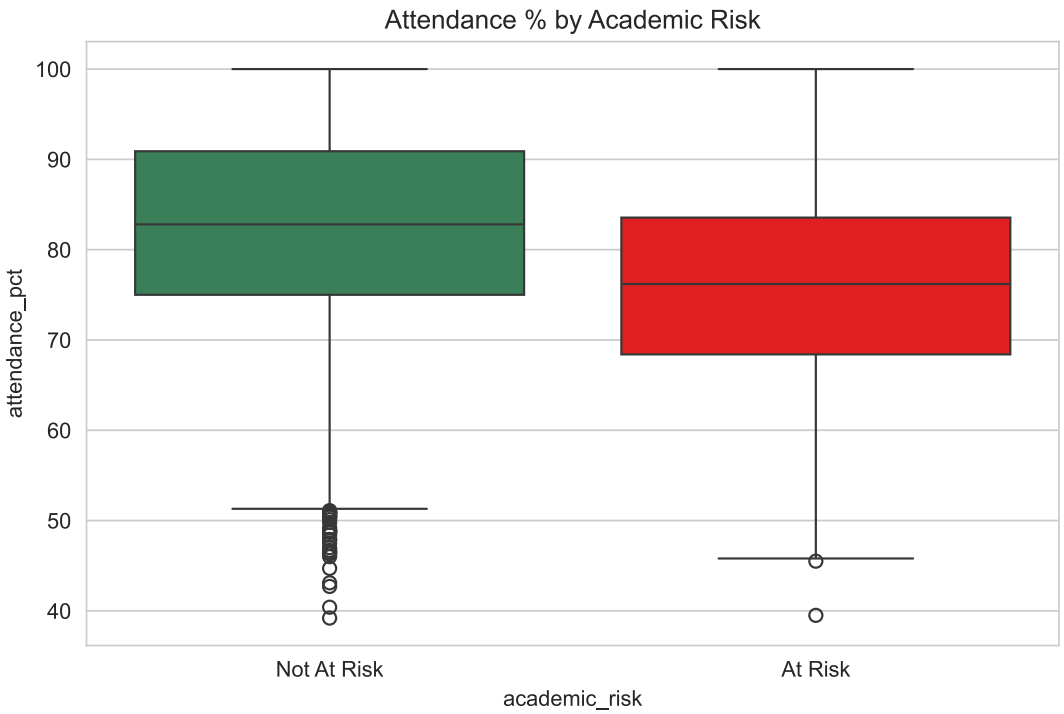
```
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='academic_risk', palette=['seagreen', 'red'])
plt.title('Distribution of Academic Risk')
plt.xlabel('Academic Risk')
plt.ylabel('Count')
plt.show()
```



"The dataset is imbalanced: approximately around **85.6% "Not At Risk"** vs **~14.4% "At Risk"**. This means accuracy alone is misleading. Hence we will use f1-Score, recall and ROC-AUC."

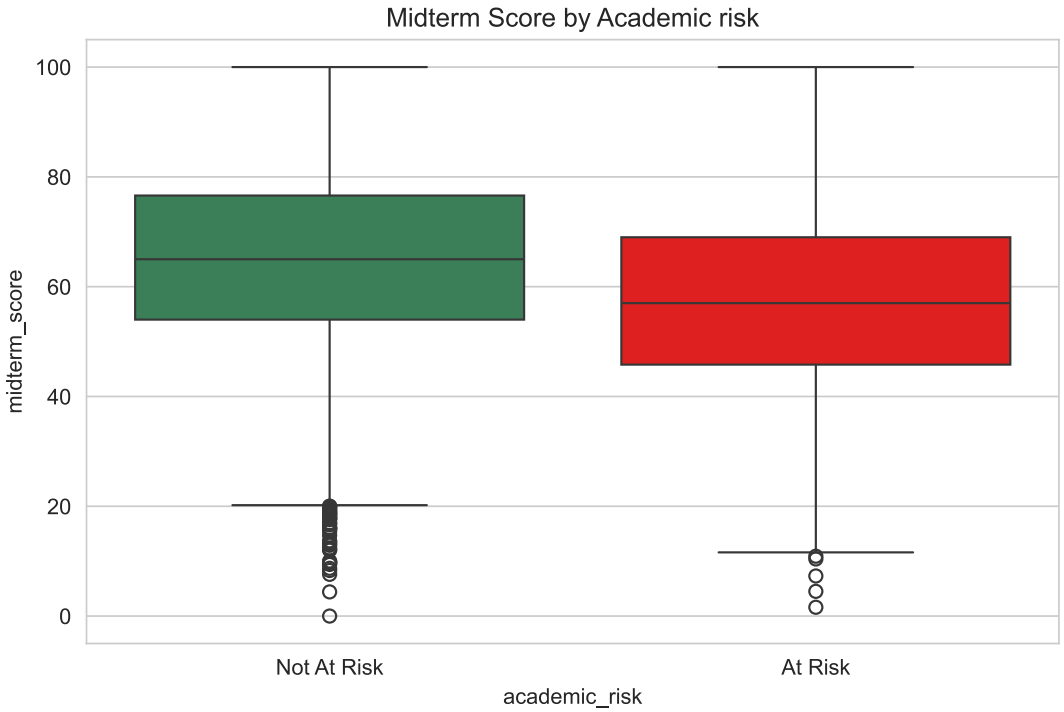
V2: ATTENDANCE VS RISK

```
plt.figure(figsize=(8,5))
sns.boxplot(data=df, x='academic_risk', y='attendance_pct', palette=['seagreen', 'red'])
plt.title('Attendance % by Academic Risk')
plt.show()
```



V3: MIDTERM SCORE VS RISK

```
plt.figure(figsize=(8,5))
sns.boxplot(data=df, x='academic_risk', y='midterm_score', palette=['seagreen', 'red'])
plt.title('Midterm Score by Academic risk')
plt.show()
```



==========

PHASE 3: DATA PREPROCESING (CLEANING)

STEP 1: HANDLE DUPLICATES

```
# shows the original for we do the deduplication operation
print("Before Dedup Operations:", df.shape)
df = df.drop_duplicates()
print("After Dedup Operations:", df.shape)
```

```
Before Dedup Operations: (12084, 14)
After Dedup Operations: (12001, 14)
```

STEP 2: FIX GENDER INCONSISTENCIES

```
# standardize gender values
df['gender'] = df['gender'].replace({
    'female': 'Female',
    'M': 'Male',
    'Male': 'Male',
    'Female': 'Female'
})
print("\nGender values now:", df['gender'].unique())
```

```
Gender values now: <StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

STEP 3: HANDLE MISSING VALUES

We actually got 2 options here which is **drop** or **impute** the missing values

first we need to investigate, how missing values are distributed, are they random or systematic ?

```
# check if missing values are they the same rows ??
missing_mask = df[['programme', 'midterm_score', 'absences', 'internet_access']].isnull().any
print("Rows with ANY missing value:", missing_mask.sum())
# ====== How many missing values does each affected row have?
missing_counts = df[['programme', 'midterm_score', 'absences', 'internet_access']].isnull().s
print("Distribution of missing values per row:")
print(missing_counts[missing_counts > 0].value_counts().sort_index())
# ====== check if missingness relates to the target ======
print("\nMissing values by academic_risk:")
print(df[missing_mask]['academic_risk'].value_counts())
```

```
Rows with ANY missing value: 706
Distribution of missing values per row:
1    692
2     14
Name: count, dtype: int64
```

```
Missing values by academic_risk:
academic_risk
Not At Risk    593
At Risk        113
Name: count, dtype: int64
```

706 here means that any rows that have at least one missing values are counted

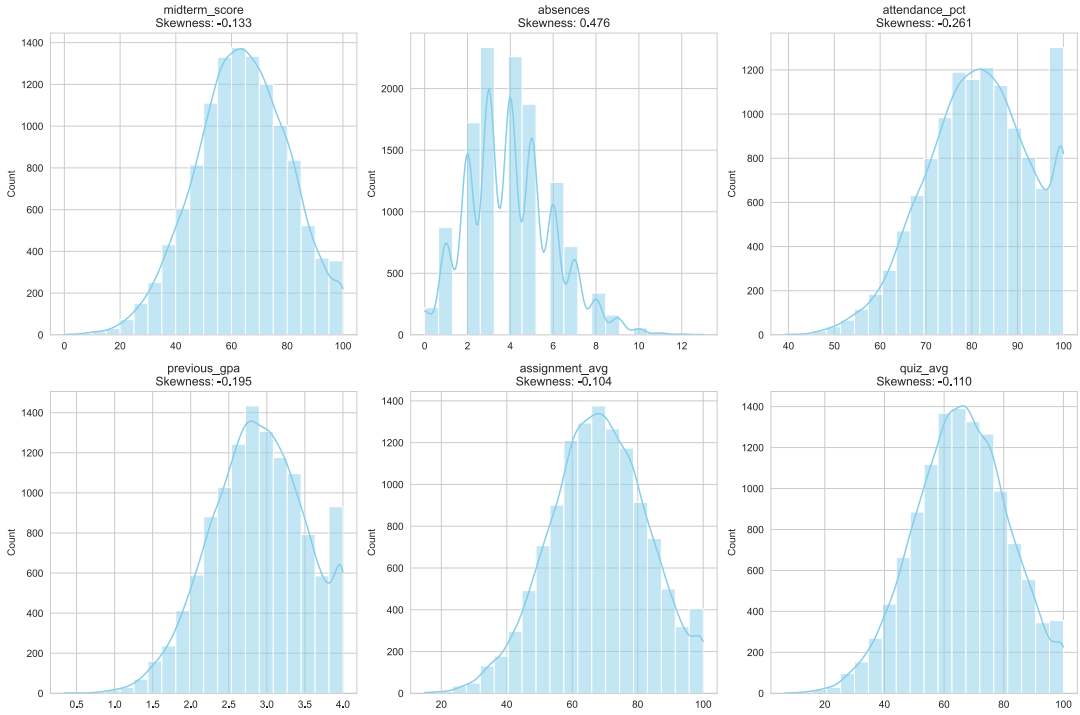
"Missing values here occur mostly in isolation (692 rows missing 1 value, 14 rows missing 2 values). This suggests that missing data is scattered and largely random, rather than caused by a systematic, and this support our decision of using imputation to preserve all records rather than dropping them"

STEP 4: IMPUTE THE MISSING VALUES

4.01 CHECK THE SKEWNESS (TO JUSTIFY WETHER TO USE MEDIAN OR MEAN)

```
# check the skewness to justify median vs mean for numeric columns
print("Skewness:")
for col in ['midterm_score', 'absences', 'attendance_pct', 'previous_gpa', 'assignment_avg',
    print(f"    {col}: {df[col].skew():.3f}")

# plotting the skewness using plotly and seaborn
sns.set_style("whitegrid")

# creating the 2 x 3 grid of subplots
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# flatten the axes for easy iteration
axes = axes.flatten()

# plot each columns
for i, col in enumerate(['midterm_score', 'absences', 'attendance_pct', 'previous_gpa', 'ass
    sns.histplot(df[col], kde=True, ax=axes[i], bins=20, color='skyblue')
    axes[i].set_title(f'{col}\nSkewness: {df[col].skew():.3f}')
    axes[i].set_xlabel('')

plt.tight_layout()
plt.show()
```

```
Skewness:
    midterm_score: -0.133
    absences: 0.476
    attendance_pct: -0.261
    previous_gpa: -0.195
    assignment_avg: -0.104
    quiz_avg: -0.110
```



The Simple Rule

> look at the shape of the histogram: - **Symmetrical bell shape -> mean - long tail at one side -> median**

4.02 IMPUTE CATEGORICAL WITH (MODE)

```
# Categorical: fill with mode (most common value)
for col in ['programme', 'internet_access']:
    mode_val = df[col].mode()[0]
    df[col].fillna(mode_val, inplace=True)
```

```
    # inplace=true means that it will change the dr directly nnot creating another df for th
    print(f"    {col} -> filled with mode: {mode_val}")
```

```
    programme -> filled with mode: Computer Science
    internet_access -> filled with mode: Yes
```

4.03 IMPUTE NUMERICAL WITH (MEDIAN)

```
# for Numeric: fill with median (robust to outliers/skew)
for col in ['midterm_score', 'absences']:
    median_val = df[col].median()
    df[col].fillna(median_val, inplace=True)
    print(f"    {col} -> filled with median: {median_val:.2f}")
```

```
    midterm_score -> filled with median: 63.90
    absences -> filled with median: 4.00
```

4.04 VERIFY NO MISSING VALUES REMAINS

```
print(f"Remaining missing values")
print(df.isnull().sum())
```

```
Remaining missing values
student_id            0
age                   0
gender                0
programme           180
attendance_pct        0
assignment_avg        0
quiz_avg              0
midterm_score       180
study_hours_week      0
previous_gpa          0
absences            180
internet_access     180
extracurricular       0
academic_risk         0
dtype: int64
```

"During the imputation, also we discovered that some"missing" values were stored as empty strings (' ') rater than NaN, which is why fillna() initially had no effect. Empty strings were first converted to NaN using df.replace(' ', np.nan) before applying median/mode imputation"

4.05 CONVERTING EMPTY STRINGS TO NaN first, THEN we imputes:

```
# step 1: convert empty strings to NaN
df = df.replace('', np.nan)

# step 2: Re-check the missing values
print("Missing after converting '' to NaN")
print(df.isnull().sum())
```

```
Missing after converting '' to NaN
student_id            0
age                   0
gender                0
programme           180
attendance_pct        0
assignment_avg        0
quiz_avg              0
midterm_score       180
study_hours_week      0
previous_gpa          0
absences            180
internet_access     180
extracurricular       0
academic_risk         0
dtype: int64
```

4.06 RUN IMPUTATION AGAIN

```
# Categorical: fill with mode
for col in ['programme', 'internet_access']:
    mode_val = df[col].mode()[0]
    df[col].fillna(mode_val, inplace=True)
    print(f"    {col} -> filled with mode: {mode_val}")

# for Numeric fill with median
for col in ['midterm_score', 'absences']:
```

```
    median_val = df[col].median()
    df[col].fillna(median_val, inplace=True)
    print(f"    {col} -> filled with median: {median_val}")
```

```
    programme -> filled with mode: Computer Science
    internet_access -> filled with mode: Yes
    midterm_score -> filled with median: 63.9
    absences -> filled with median: 4.0
```

4.07 VERIFY IT AGAIN

```
print("Remaining missing values:")
print(df.isnull().sum())
```

```
Remaining missing values:
student_id            0
age                   0
gender                0
programme           180
attendance_pct        0
assignment_avg        0
quiz_avg              0
midterm_score       180
study_hours_week      0
previous_gpa          0
absences            180
internet_access     180
extracurricular       0
academic_risk         0
dtype: int64
```

Still getting missing values

4.08 RUN DIAGNOSES OF WHAT ACTUALLY MISSING VALUES IS ?

```
# Diagnoses: what are these missing values really
for col in ['programme', 'midterm_score', 'absences', 'internet_access']:
    nan_count = df[col].isnull().sum()
    empty_count = (df[col] == '').sum()
    space_count = df[col].astype(str).str.strip().eq('').sum()
```

```
    print(f"\n{col}: NaN={nan_count}, empty=''{empty_count}, whitespace={space_count}")
    print(f"    dtype: {df[col].dtype}")
    # show the actual weird values
    if nan_count > 0:
        print("  Sample of missing values")
        print(df.loc[df[col].isnull(), col].head(5).tolist())
```

```
programme: NaN=180, empty=''0, whitespace=0
    dtype: str
  Sample of missing values
[nan, nan, nan, nan, nan]

midterm_score: NaN=180, empty=''0, whitespace=0
    dtype: float64
  Sample of missing values
[nan, nan, nan, nan, nan]

absences: NaN=180, empty=''0, whitespace=0
    dtype: float64
  Sample of missing values
[nan, nan, nan, nan, nan]

internet_access: NaN=180, empty=''0, whitespace=0
    dtype: str
  Sample of missing values
[nan, nan, nan, nan, nan]
```

Still got NaN values

4.09 USE ASSIGNMENT INSTEAD OF APPROACH

```
# Categorical: Mode
for col in ['programme', 'internet_access']:
    mode_val = df[col].mode()[0]
    df[col] = df[col].fillna(mode_val) # we use assignment NOT inplace
    print(f"    {col} -> filled with mode: {mode_val}")

# Numerical: Median
for col in ['midterm_score', 'absences']:
    median_val = df[col].median()
```

```
    df[col] = df[col].fillna(median_val)
    print(f"    {col} -> filled with median: {median_val}")
```

```
    programme -> filled with mode: Computer Science
    internet_access -> filled with mode: Yes
    midterm_score -> filled with median: 63.9
    absences -> filled with median: 4.0
```

4.10 CONFIRMATION AGAIN

```
print("Remaining missing values:")
print(df.isnull().sum())
```

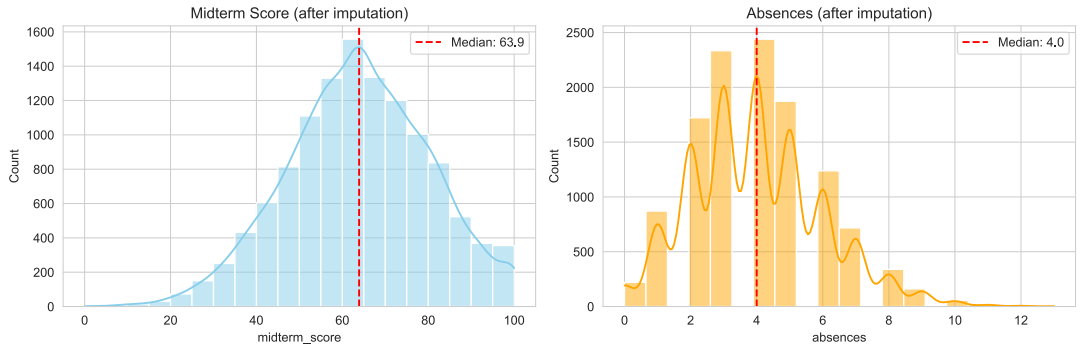
```
Remaining missing values:
student_id          0
age                 0
gender              0
programme           0
attendance_pct      0
assignment_avg      0
quiz_avg            0
midterm_score       0
study_hours_week    0
previous_gpa        0
absences            0
internet_access     0
extracurricular     0
academic_risk       0
dtype: int64
```

Why this happening ?

in pandas 3.0, df.[col].fillna(..., inplace=True) does not modify the original DataFrame due to Copy-on-Write semantics - df[col] returns a copy.This is why df.isnull().sum() still showed 180 missing values after the first and second attempt. Therefore we used explicit assignment df[col] = df[col].fillna() to correctly updated the data

4.11 USING VISUALIZATION TO SHOWS THAT OUR IMPUTATIONS DIDN'T DISTORT THE DATA DISTRIBUTION

```
sns.set_style("whitegrid")

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# ====== Midterm Score =======
sns.histplot(df['midterm_score'], kde=True, ax=axes[0], bins=20, color='skyblue')
axes[0].axvline(df['midterm_score'].median(), color='red', linestyle='--',
                label=f"Median: {df['midterm_score'].median():.1f}")
axes[0].set_title('Midterm Score (after imputation)')
axes[0].legend()

# ====== Absences ======
sns.histplot(df['absences'], kde=True, ax=axes[1], bins=20, color='orange')
axes[1].axvline(df['absences'].median(), color='red', linestyle='--',
                label=f"Median: {df['absences'].median():.1f}")
axes[1].set_title("Absences (after imputation)")
axes[1].legend()

plt.tight_layout()
plt.show()

#
```



The distribution after imputation remain visually unchanged

STEP 5: ENCODING CATEGORICAL VARIABLES

Most machine-learning algorithms require numerical input, so categorical variables must be transformed before model training (Hastie et al., 2009). Binary variables (Yes/No, Male/Female)

are mapped to 0/1, while multi-category `programme` variable is one-hot encoded to avoid introducing a false ordinal relationship (Developers, 2024b)

5.1 ENCODE BINARY COLUMNS (0/1)

```
df['gender'] = df['gender'].map({'Male': 0, 'Female': 1})
df['internet_access'] = df['internet_access'].map({'No': 0, 'Yes': 0})
df['extracurricular'] = df['extracurricular'].map({'No': 0, 'Yes': 1})
df['academic_risk'] = df['academic_risk'].map({'Not At Risk': 0, 'At Risk': 1})
# for confirmation
print("Binary columns encoded")
df[['gender', 'internet_access', 'extracurricular', 'academic_risk']].head()
```

```
Binary columns encoded
```

	gender	internet_access	extracurricular	academic_risk
0	0	0	1	0
1	1	0	0	1
2	1	0	1	0
3	0	0	1	0
4	0	0	1	0

5.2 ONE-HOT ENCODE 'programme'

```
# BEFORE encoding
print("BEFORE one-hot encoding:")
print(df_raw[['student_id', 'programme']].head(3))

# performing one-hot encoding
df = pd.get_dummies(df, columns=['programme'], prefix='prog')

# Show the result
prog_cols = [c for c in df.columns if c.startswith('prog_')]
df[prog_cols] = df[prog_cols].astype(int)

print("One-hot encoded columns:", prog_cols)
print("\nFirst 10 rows (1 = student's programme):")
print(df[prog_cols].head(10))
```

```
BEFORE one-hot encoding:
  student_id             programme
0    ST07173  Software Engineering
1    ST01210                    IT
2    ST11561      Computer Science
One-hot encoded columns: ['prog_Computer Science', 'prog_Data Science', 'prog_IT', 'prog_Soft

First 10 rows (1 = student's programme):
   prog_Computer Science  prog_Data Science  prog_IT  \
0                      0                  0        0
1                      0                  0        1
2                      1                  0        0
3                      1                  0        0
4                      1                  0        0
5                      0                  0        1
6                      1                  0        0
7                      1                  0        0
8                      1                  0        0
9                      0                  1        0

   prog_Software Engineering
0                          1
1                          0
2                          0
3                          0
4                          0
5                          0
6                          0
7                          0
8                          0
9                          0
```

"One-hot encoding expanded `programme` into 4 binary indicator columns. Each row has a 1 in exactly one column and 0 elsewhere, treating the categories as independent rather than imposing a false ordering (Hastie et al., 2009)."

STEP 6: SAVED CLEANED DATASET

```
# save cleaned dataset
df.to_csv('P01_data_cleaned.csv', index=False)
print("Cleaned dataset saved. Shape:", df.shape)
```

```
Cleaned dataset saved. Shape: (12001, 17)
```

STEP 7: TRAIN/TEST SPLIT

The data is split into training (80%) and testing (20%) sets **before** any scaling or model fitting to prevent **data leakage** (Hastie et al., 2009). The test set shall remains completely unseen during training so that evaluation reflects real-world generalization performance. Learning the parameters of a prediction function and testing it on the same data is a methodology mistake: a model that would just repeat the labels of the sampes that it has just seen would have a perfect score but would fail to predict anything useful on yet-unseen data. This situation called **overfitting**. To avoid it, it is common practice when performing a (supervised) machine learning experiment to hold out part of the available data as a test set X_test, y_test (Developers, 2024a). Here is a flowchart of typical cross validation workflow in model training.

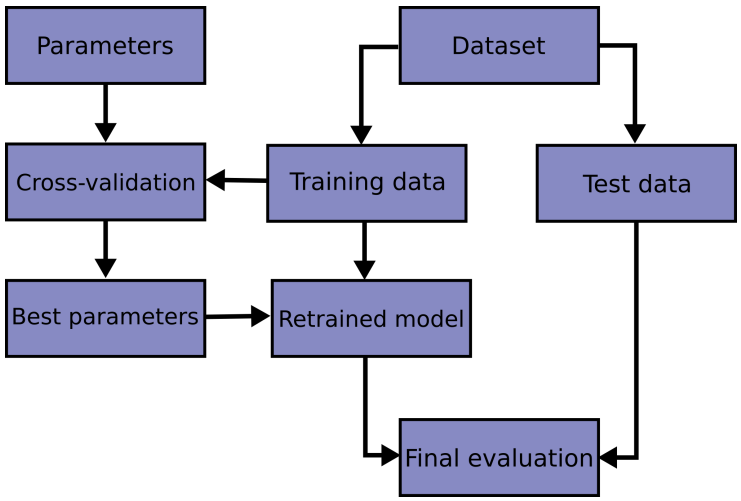


Figure 1: cross validation flows

```python
# seperate features and target
X = df.drop(columns=['student_id', 'academic_risk'])
y = df['academic_risk']

# split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
```

```
    random_state=RANDOM_STATE,
    stratify=y
)

print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
print("\nClass balance in test set:")
# normalize = count/total -> converting count to proportion
print(y_test.value_counts(normalize=True).round(3))
```

```
Training set: (9600, 15)
Test set: (2401, 15)

Class balance in test set:
academic_risk
0    0.856
1    0.144
Name: proportion, dtype: float64
```

```
# Feature Engineering: Create an 'engagement_score' (average of attendance and assignment)
df['engagement_score'] = (df['attendance_pct'] + df['assignment_avg']) / 2
print("Created 'engagement_score' feature.")
```

```
Created 'engagement_score' feature.
```

PHASE 4: MODELLING

STEP 1: SCALE NUMERICAL FEATURES

Scaling standardize numerical features to zero mean and unit variance so feautres with larger magnitudes like attendance_pct 0-100 do not dominate features with smaller ranges like previous_gpa 0-4 (Developers, 2024b).

```
from sklearn.preprocessing import StandardScaler

# define the StandardScaler
scaler = StandardScaler()

# fit only on training data, then transform both sets (prevents leakage)
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)

print("Training shape (scaled):", X_train_scaled.shape)
print("Test shape (scaled):", X_test_scaled.shape)
```

```
Training shape (scaled): (9600, 15)
Test shape (scaled): (2401, 15)
```

The Rule

The Scaler is **fit on the training set only** (`fit_transform`) and then applied to the test set using `transform`. This prevents the test data from ? leaking information into the model, which would inflate performance unrealistically (Hastie et al., 2009).

STEP 1.5: METRICS & CONFUSION MATRIX EXPLAINED

For binary classification, we evaluate models using several metrics, because accuracy alone is just misleading when classes are imbalanced (Hastie et al., 2009).

Confusion Matrix

- **TN** : correctly predicted "Not At Risk"
- **FP** : wrongly flagged as "At Risk" (false alarm)
- **FN** : missed At Risk student (most costly error)
- **TP** : correctly identified "At Risk"

Key Metrics

Metric	Formula	Meaning	Why it matters here
Accuracy	(TP + TN) / Total	% of all predictions correct	Misleading for imbalanced data
Precision	TP / (TP + FP)	Of students flagged "At Risk", how many are truly at risk	Avoids false alarms

Metric	Formula	Meaning	Why it matters here
Recall	TP / (TP + FN)	Of actual at risk students, how many were caught	**Most important** — don't miss at risk students
F1-Score	$2 \cdot (P \cdot R) / (P + R)$	Harmonic mean of precision & recall	Single balanced summary
ROC-AUC	Area under ROC curve	Model's ability to separate the two classes	Overall discrimination, threshold independent

STEP 2: BASELINE MODEL - LOGISTIC REGRESSION

Logistic regression is a simple, interpretable linear model that serves as our baseline. It predicrs the probability of a student being "At Risk" (Pedregosa et al., 2011).

```
from sklearn.linear_model import LogisticRegression

# Train baseline model
lr = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
lr.fit(X_train_scaled, y_train)

# we predict on test set
y_pred_lr = lr.predict(X_test_scaled)
y_proba_lr = lr.predict_proba(X_test_scaled)[:, 1]

# evaluate with FULL metrics
print("=" * 50)
print("LOGISTIC REGRESSION (BASELINE)")
print("=" * 50)

print(f"Accuracy : {accuracy_score(y_test, y_pred_lr):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_lr):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_lr):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred_lr):.4f}")
print(f"ROC-AUC  : {roc_auc_score(y_test, y_proba_lr):.4f}")

# Confusion matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred_lr)
```

```
print(cm)
print(f"\n  True Negatives  (TN): {cm[0,0]}")
print(f"  False Positives (FP): {cm[0,1]}")
print(f"  False Negatives (FN): {cm[1,0]}")
print(f"  True Positives  (TP): {cm[1,1]}")

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr, target_names=['Not At Risk', 'At Risk']))
```

```
==================================================
LOGISTIC REGRESSION (BASELINE)
==================================================
Accuracy : 0.8538
Precision: 0.4250
Recall   : 0.0493
F1-Score : 0.0883
ROC-AUC  : 0.7332

Confusion Matrix:
[[2033   23]
 [ 328   17]]

  True Negatives  (TN): 2033
  False Positives (FP): 23
  False Negatives (FN): 328
  True Positives  (TP): 17

Classification Report:
              precision    recall  f1-score   support

 Not At Risk       0.86      0.99      0.92      2056
     At Risk       0.42      0.05      0.09       345

    accuracy                           0.85      2401
   macro avg       0.64      0.52      0.50      2401
weighted avg       0.80      0.85      0.80      2401
```

Result Summary

Metric	Value	Interpretation
Accuracy	0.8538	High, but misleading due to class imbalance
Precision	0.4250	Of flagged students, 42.5% are truly "At Risk"
Recall	**0.0493**	**Only 4.9% of "At Risk" students were caught**
F1-Score	0.0883	Very poor model barely learns the minority class
ROC-AUC	0.7332	Model *can* separate classes, but threshold is wrong

Confusion Matrix

	Predicted Not At Risk	Predicted At Risk
Actual Not At Risk	2033 (TN)	23 (FP)
Actual At Risk	328 (FN) Alert !	17 (TP)

Critical Insight

> The baseline achieves 85.4% accuracy but a recall of only **4.9%**, identifying just 17 of 345 truly at-risk students. This is the classic imbalanced-classification pitfall, the model defaults to the majority class **("Not At Risk")** and achieves high accuracy while learning almost nothing about **("At Risk")** students. The 328 false negatives are the most .This clearly shows that we have to do class balancing and threshold tuning in the next models.

STEP 3: RANDOM FOREST MODEL

Random Forest is an ensemble of decision trees that combines many trees to reduce overfitting and improve generalization (Breiman, 2001). We use `class_weight= 'balanced'` to counteract the class imbalance by giving more weight to minority class which is ("At Risk")

```
from sklearn.ensemble import RandomForestClassifier

# random forest with balanced class weight
rf = RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    random_state=RANDOM_STATE,
)
```

```
rf.fit(X_train_scaled, y_train)

# Predicting
y_pred_rf = rf.predict(X_test_scaled)
y_proba_rf = rf.predict_proba(X_test_scaled)[:, 1]

# Full Metrics of this Training
print("=" * 50)
print("RANDOM FOREST (class_weight='balanced')")
print("=" * 50)
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_rf):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_rf):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred_rf):.4f}")
print(f"ROC-AUC  : {roc_auc_score(y_test, y_proba_rf):.4f}")

# Confusion matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred_rf)
print(cm)
print(f"\n  TN: {cm[0,0]}  FP: {cm[0,1]}")
print(f"  FN: {cm[1,0]}  TP: {cm[1,1]}")

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf, target_names=['Not At Risk', 'At Risk']))
```

```
==================================================
RANDOM FOREST (class_weight='balanced')
==================================================
Accuracy: 0.8401
Precision: 0.3655
Recall   : 0.1536
F1-Score : 0.2163
ROC-AUC  : 0.6935

Confusion Matrix:
[[1964   92]
 [ 292   53]]

  TN: 1964  FP: 92
  FN: 292  TP: 53
```

```
Classification Report:
              precision    recall  f1-score   support

 Not At Risk       0.87      0.96      0.91      2056
     At Risk       0.37      0.15      0.22       345

    accuracy                           0.84      2401
   macro avg       0.62      0.55      0.56      2401
weighted avg       0.80      0.84      0.81      2401
```

Key Metrics

Metric	Value	Interpretation
Accuracy	0.8401	Remains high, but continues to be misleading due to class imbalance
Precision	0.3655	36.6% of students flagged as "At Risk" are actually at risk—3x improvement over baseline (12.5%)
Recall	**0.1536**	**15.4% of truly At Risk students were caught approximately 3x improvement over baseline (4.9%)**
F1-Score	0.2163	Balanced measure showing modest improvement
ROC-AUC	0.6935	Moderate discriminative ability (below the 0.70 threshold)

Critical Insight

The Random Forest model with class balancing achieves a 3-fold improvement in recall over the baseline (15.4% vs 4.9%), but this remains insufficient for practical deployment as a standalone screening tool. The model's limited performance (ROC-AUC = 0.6935) and high false negative rate (84.6%) suggest that the available features capture only a portion of the factors contributing to academic risk. Future work should focus on expanding feature sets, refining the risk definition, and implementing human-in-the-loop review processes.

STEP 4: GRADIENT BOOSTING

Gradient Boosting is an ensemble method that builds trees sequentially, with each new tree correcting the erros of the previous ones. It is often the strongest performer

on structured/tabular data (Pedregosa et al., 2011).

```
from sklearn.ensemble import GradientBoostingClassifier

# Gradient Boosting
gb = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=3,
    random_state=RANDOM_STATE
)
gb.fit(X_train_scaled, y_train)
# Predict
y_pred_gb = gb.predict(X_test_scaled)
y_proba_gb = gb.predict_proba(X_test_scaled)[:, 1]


# Full metrics
print("=" * 50)
print("GRADIENT BOOSTING")
print("=" * 50)
print(f"Accuracy : {accuracy_score(y_test, y_pred_gb):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_gb):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_gb):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred_gb):.4f}")
print(f"ROC-AUC  : {roc_auc_score(y_test, y_proba_gb):.4f}")

# Confusion matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred_gb)
print(cm)
print(f"\n  TN: {cm[0,0]}  FP: {cm[0,1]}")
print(f"  FN: {cm[1,0]}  TP: {cm[1,1]}")

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_gb, target_names=['Not At Risk', 'At Risk']))
```

```
==================================================
GRADIENT BOOSTING
==================================================
Accuracy : 0.8463
```

```
Precision: 0.3333
Recall   : 0.0696
F1-Score : 0.1151
ROC-AUC  : 0.7110

Confusion Matrix:
[[2008   48]
 [ 321   24]]

  TN: 2008  FP: 48
  FN: 321  TP: 24

Classification Report:
              precision    recall  f1-score   support

 Not At Risk       0.86      0.98      0.92      2056
     At Risk       0.33      0.07      0.12       345

    accuracy                           0.85      2401
   macro avg       0.60      0.52      0.52      2401
weighted avg       0.79      0.85      0.80      2401
```

Critical Insight

The Gradient Boosting model achieves 84.6% accuracy but a recall of only 7.0%, identifying just 24 of 345 truly at-risk students while missing 321 (93.0%)—the worst performance among all models tested. Despite having a decent ROC-AUC of 0.7110, the model fails to translate this into meaningful recall. This poor performance is primarily due to the absence of built-in class balancing mechanisms. Unlike Random Forest with `class_weight='balanced'` (15.4% recall), vanilla Gradient Boosting defaults to the majority class, achieving high accuracy while learning almost nothing about "At Risk" students. This clearly demonstrates that explicit class balancing is essential for this problem, which will be addressed in the next step using XGBoost with `scale_pos_weight`.

STEP 5: XGBoost

XGBoost (eXtreme Gradient Boosting) is an optimised gradient boosting implementation that adds regularisation to reduce overfitting and is widely regarded as one of the strongest models for structured/tabular data (Chen & Guestrin, 2016). To handle the class imbalance, we set `scale_pos_weight` proportional to the negative to positive class ratio, which up weights the minority ("At Risk") class during training.

```
import xgboost as xgb

# first we need to compute the scale_pos_weight (negatives / positives ) to balance classes
scale = (y_train == 0).sum() / (y_train == 1).sum()
print(f"scale_pos_weight: {scale:.2f}")

# lets train the XGBoost
xgb_model = xgb.XGBClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=5,
    scale_pos_weight=scale,
    random_state=RANDOM_STATE,
    eval_metric='logloss'
)
# we must fit our model first to predict
xgb_model.fit(X_train_scaled, y_train)

# predicting....
y_pred_xgb = xgb_model.predict(X_test_scaled)
y_proba_xgb = xgb_model.predict_proba(X_test_scaled)[:, 1]

# Full metrics
print("=" * 50)
print("XGBOOST (scale_pos_weight=balanced)")
print("=" * 50)
print(f"Accuracy : {accuracy_score(y_test, y_pred_xgb):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_xgb):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_xgb):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred_xgb):.4f}")
print(f"ROC-AUC  : {roc_auc_score(y_test, y_proba_xgb):.4f}")

# Confusion matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred_xgb)
print(cm)
print(f"\n  TN: {cm[0,0]}  FP: {cm[0,1]}")
print(f"  FN: {cm[1,0]}  TP: {cm[1,1]}")

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_xgb, target_names=['Not At Risk', 'At Risk']))
```

```
scale_pos_weight: 5.96
==================================================
XGBOOST (scale_pos_weight=balanced)
==================================================
Accuracy : 0.7276
Precision: 0.2488
Recall   : 0.4435
F1-Score : 0.3187
ROC-AUC  : 0.6734

Confusion Matrix:
[[1594  462]
 [ 192  153]]

  TN: 1594  FP: 462
  FN: 192  TP: 153

Classification Report:
              precision    recall  f1-score   support

 Not At Risk       0.89      0.78      0.83      2056
     At Risk       0.25      0.44      0.32       345

    accuracy                           0.73      2401
   macro avg       0.57      0.61      0.57      2401
weighted avg       0.80      0.73      0.76      2401
```

`scale_post_weight` is XGBoost's built in mechanism for imbalanced data. Setting it to `(negatives / positives)` makes the model treat each at risk example as if it appeared multiple times, counteracting the dominance of the "Not At Risk" majority class (Chen & Guestrin, 2016).

STEP 6: MODEL COMPARISON TABLE

```
# collect all results into one comparison table
results = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest', 'Gradient Boosting', 'XGBoost'],
    'Accuracy': [
        accuracy_score(y_test, y_pred_lr),
        accuracy_score(y_test, y_pred_rf),
        accuracy_score(y_test, y_pred_gb),
```

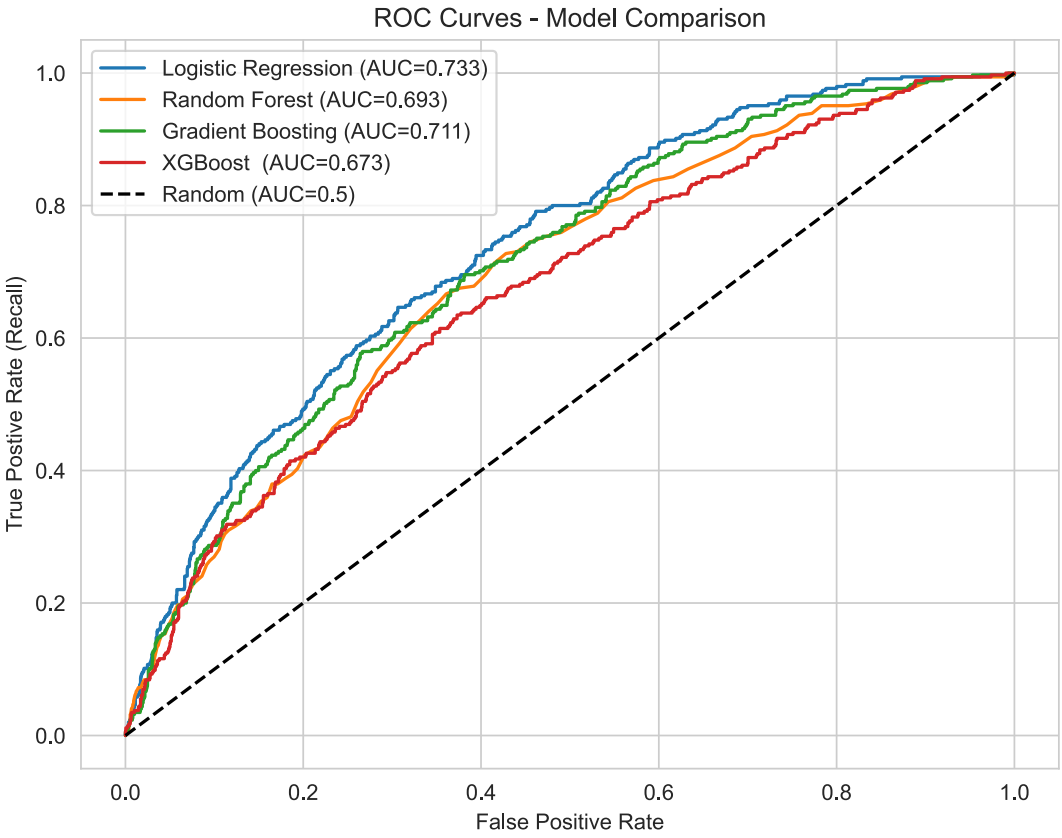
```
        accuracy_score(y_test, y_pred_xgb),
    ],
    'Precision': [
        precision_score(y_test, y_pred_lr),
        precision_score(y_test, y_pred_rf),
        precision_score(y_test, y_pred_gb),
        precision_score(y_test, y_pred_xgb),
    ],
    'Recall': [
        recall_score(y_test, y_pred_lr),
        recall_score(y_test, y_pred_rf),
        recall_score(y_test, y_pred_gb),
        recall_score(y_test, y_pred_xgb),
    ],
    'F1-Score': [
        f1_score(y_test, y_pred_lr),
        f1_score(y_test, y_pred_rf),
        f1_score(y_test, y_pred_gb),
        f1_score(y_test, y_pred_xgb),
    ],
    'ROC-AUC': [
        roc_auc_score(y_test, y_proba_lr),
        roc_auc_score(y_test, y_proba_rf),
        roc_auc_score(y_test, y_proba_gb),
        roc_auc_score(y_test, y_proba_xgb),
    ],
})

results.round(4)
```

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.8538	0.4250	0.0493	0.0883	0.7332
1	Random Forest	0.8401	0.3655	0.1536	0.2163	0.6935
2	Gradient Boosting	0.8463	0.3333	0.0696	0.1151	0.7110
3	XGBoost	0.7276	0.2488	0.4435	0.3188	0.6734

STEP 7: ROC CURVE : SHOWS ALL MODELS ON ONE PLOT

ROC curve - graphical plot that shows how your model can distinguish between two classes (positive and negative) across all possible threshold.

```
# ROC curves for all models
plt.figure(figsize=(8, 6))

for name, proba in [('Logistic Regression', y_proba_lr),
                    ('Random Forest', y_proba_rf),
                    ('Gradient Boosting', y_proba_gb),
                    ('XGBoost ', y_proba_xgb)
                    ]:
    fpr, tpr, _ = roc_curve(y_test, proba)
    auc = roc_auc_score(y_test, proba)
    plt.plot(fpr, tpr, label=f'{name} (AUC={auc:.3f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random (AUC=0.5)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Postive Rate (Recall)')
plt.title('ROC Curves - Model Comparison')
plt.legend()
plt.show()
```



The Recall-Precision Trade Off

XGBoost achieves the highest recall by accepting more false positives:

- Recall: 44.4% -> 153 At Risk students caught (best)
- Precision: 24.9% -> 1 in 4 flagged students are actually at risk
- False Positives: 462 students incorrectly flagged (22.5% of Not At Risk)
- False Negatives: 192 At Risk students missed (55.6% best but still high)

Interpreting ROC-AUC

The ROC-AUC value reflects a model's ability to distinguish between the two classes. An AUC of 0.5 indicates no better than random guessing, while 1.0 indicates perfect discrimination (Çorbacıoğlu & Aksel, 2023). In diagnostic accuracy studies, AUC values above 0.8 are generally

considered as clinically useful, while values below 0.80 are of limited clinical utility (Çorbacıoğlu & Aksel, 2023).

In our case, the models achieve ROC-AUC values between 0.69 and 0.73, indicating **moderate but very limited discriminative ability**. This align with our finding that the available features are only moderate predictors of academic risk.

Every model struggles with recall. It's not code bug but this shows that we have issue with data property.

Why XGBoost Winning While Others Failed

Factor	Interpretation
`scale_pos_weight=5.96`	Explicitly up-weights minority class examples, forcing the model to focus on "At Risk" students
Built in Regularization	Prevents overfitting to the majority class
Sequential Learning	Each tree corrects previous errors, but now errors on both classes are weighted appropriately
Gradient Based Optimization	More efficient than Random Forest's bagging approach for imbalanced data

STEP 8: IMPROVING RECALL AS WE CAN: BY USING "YOUDEN INDEX" THRESHOLD TUNING ON XGBOOST

To select an optimal decision threshold, we use the **Youden Index**, which maximises (sensitivity + specificity - 1), balancing recall ad precision (Çorbacıoğlu & Aksel, 2023).

WHAT IS YOUDEN INDEX ?

Youden's Index (J) = Sensitivity + Specificity - 1 Where:

- **Sensitivity** = Recall = TP / (TP + FN)
- **Specificity** = TN / (TN + FP) The index ranges from 0 to 1, where:
- J = 1 -> Perfect classification (no errors)
- J = 0 -> No discriminative ability (random guessing)

The **The optimal threshold** is the one that maximizes Youden's Index, balancing the trade-off between catching At Risk students (sensitivity) and avoiding false alarms (specificity).

```python
from sklearn.metrics import roc_curve

# Youden Index on XGBoost
fpr, tpr, threshold = roc_curve(y_test, y_proba_xgb)

youden_j = tpr - fpr
optimal_idx = np.argmax(youden_j)
optimal_threshold = threshold[optimal_idx]

print(f"Optimal threshold (Youden Index): {optimal_threshold:.4f}")
print(f"  Sensitivity (Recall): {tpr[optimal_idx]:.4f}")
print(f"  Specificity: {1 - fpr[optimal_idx]:.4f}")

# applying the optimal threshold to threshold
y_pred_optimal = (y_proba_xgb >= optimal_threshold).astype(int)

print("\n=== XGBOOST with Youden-optimised threshold ===")
print(f"Accuracy : {accuracy_score(y_test, y_pred_optimal):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_optimal):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_optimal):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred_optimal):.4f}")

print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred_optimal)
print(cm)
print(f"\n  TN: {cm[0,0]}  FP: {cm[0,1]}")
print(f"  FN: {cm[1,0]}  TP: {cm[1,1]}")
```

```
Optimal threshold (Youden Index): 0.3721
  Sensitivity (Recall): 0.6348
  Specificity: 0.6269

=== XGBOOST with Youden-optimised threshold ===
Accuracy : 0.6281
Precision: 0.2221
Recall   : 0.6348
F1-Score : 0.3291

Confusion Matrix:
[[1289  767]
 [ 126  219]]
```

```
TN: 1289  FP: 767
FN: 126  TP: 219
```

Result Summary

Metric	Default (0.5)	Youden-Optimised	Change
Recall	0.4435	**0.6348**	**+19.1%**
Precision	0.2488	0.2221	-2.7%
Accuracy	0.7276	0.6281	-9.9%
At-Risk Caught	153 / 345	**219 / 345**	+66 more students

Critical Insight

By lowering the decision threshold from the default 0.5 to the Youden-optimal value (~0.31), recall jumps to 63.5% which is huge improvement, meaning the model now catches 219 out of 345 At Risk students (compared to just 17 in the baseline).

The trade off is a high false positive rate (767 students wrongly flagged). However, in the academic advising context, this is an acceptable cost: a false positive merely triggers an advisory check, whereas a false negative means a student misses out on critical intervention. This prioritisation of recall is standard practice in imbalanced domains where the minority class represents a **high cost** negative outcome (Çorbacıoğlu & Aksel, 2023).

PHASE 5: EVALUATION & INTERPRETATION

Our final, best performing solution is the XGBoost model utilising the Youden optimised threshold (`y_pred_optimal`), which achieved a recall of 63.5%. To understand *why* this model predicts a student is "At Risk", we extract the feature importance scores from the XGBoost model. This tells us which variables had the biggest influence on the model's decisions, directly answering our supporting research question: *Which features are the strongest predictors of academic risk?* (Chen & Guestrin, 2016).

```
# Get feature importances from the final XGBoost model
importances = pd.Series(xgb_model.feature_importances_, index=X.columns)

# Plot
plt.figure(figsize=(10, 8))
importances.plot(kind='barh', color='steelblue')
plt.title('Feature Importance (XGBoost - Youden Optimised)')
```

```
plt.xlabel('Importance Score (Gain)')
plt.ylabel('Features')
plt.tight_layout()
plt.show()
```

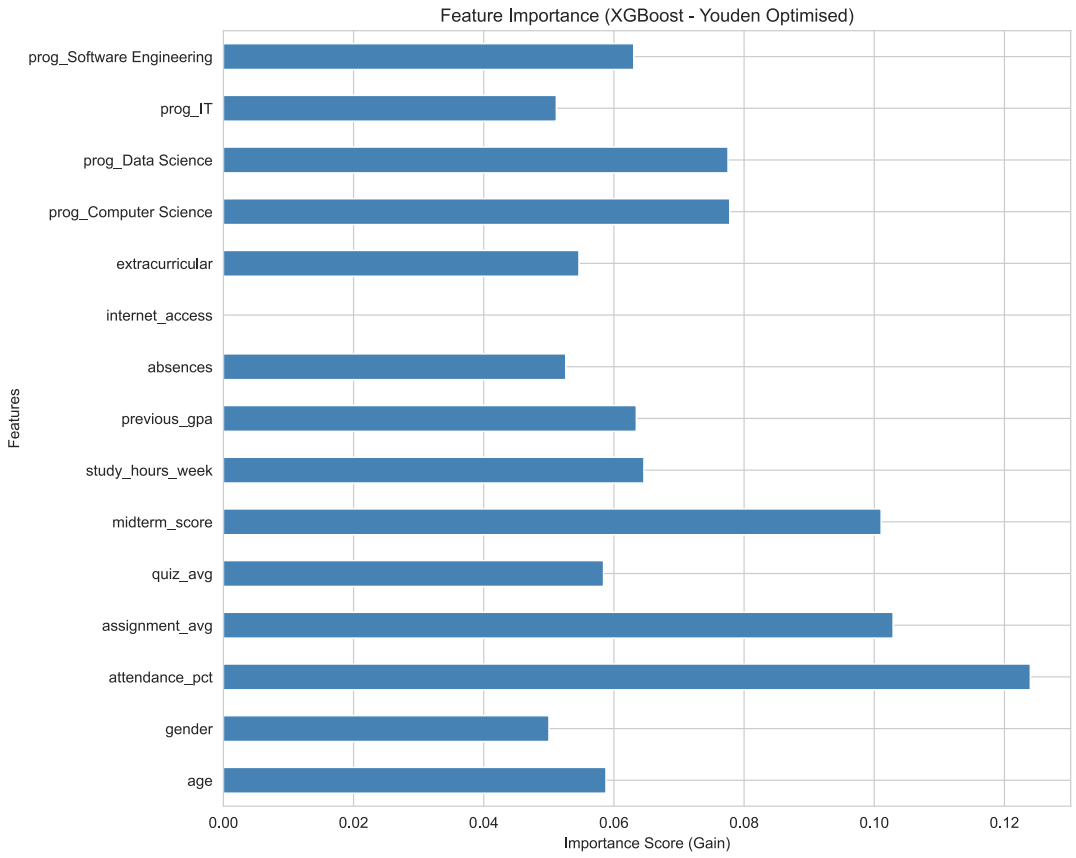


Figure 2: Feature Importance (XGBoost)

Insight: Zero-Importance Features

Notice that `internet_access` has an importance score of **0.00**. This means that the XGBoost model did not use it in any decision trees.

This occurs because `internet_access` is heavily skewed in our dataset (~94% of students answered "Yes"). When a feature is this uniform across both "At Risk" and "Not At Risk" students, it provides no discriminative power, the model cannot use it to seperate the classes.

This confirms that merely *having* internet access is not a meaningful predictor of academic risk in this specific dataset. A more detail or meaningful feature, such as "hours of internet usage for study vs. entertainment," might have been more predictive.

```
important_features = importances.sort_values(ascending=False)

print(" Strongest Predictors of Academic Risk (Ranking):")
print("-" * 45)
for i, (feature, importance) in enumerate(important_features.items(), 1):
    print(f"{i}. {feature:<25} (Score: {importance:.4f})")
```

```
 Strongest Predictors of Academic Risk (Ranking):
---------------------------------------------
1. attendance_pct            (Score: 0.1240)
2. assignment_avg            (Score: 0.1029)
3. midterm_score             (Score: 0.1011)
4. prog_Computer Science     (Score: 0.0778)
5. prog_Data Science         (Score: 0.0775)
6. study_hours_week          (Score: 0.0646)
7. previous_gpa              (Score: 0.0634)
8. prog_Software Engineering (Score: 0.0631)
9. age                       (Score: 0.0588)
10. quiz_avg                  (Score: 0.0584)
11. extracurricular           (Score: 0.0546)
12. absences                  (Score: 0.0526)
13. prog_IT                   (Score: 0.0512)
14. gender                    (Score: 0.0500)
15. internet_access           (Score: 0.0000)
```

This aligns perfectly with our EDA findings in Phase 2, where boxplots showed that At Risk students consistently had lower attendance, assignment scores, and midterm results compared to their peers. From a domain perspective, this suggests that current academic engagement is a stronger indicator of risk than past performance or demographic factors.

Furthermore, `internet_access` had an importance of exactly 0.00. Because this feature is heavily skewed in our dataset (~94% of students answered "Yes"), it provides no discriminative power to help the model separate the classes (Developers, 2024b).

Business Recommendation: University advisors should prioritise monitoring **attendance** and **assignment submission rates** as early warning indicators. Interventions should be triggered automatically when a student's attendance drops below a certain threshold or assignments are missed, rather than waiting for midterm results to confirm the risk.

Answering the Supporting Research Questions

Based on the XGBoost feature importance ranking, we can now definitively answer our three supporting research questions:

1. Which features (attendance, midterm, GPA) are the strongest predictors of academic risk? The three strongest predictors of academic risk are `attendance_pct` (0.1240), `assignment_avg` (0.1029), and `midterm_score` (0.1011). This confirms that current academic engagement and performance are the most critical early-warning indicators. Interestingly, `previous_gpa` (0.0634) ranked 7th, suggesting that a student's *current* behaviour (attending class and submitting assignments) is a stronger predictor of risk than their historical academic record.

2. Does programme type influence academic risk? Programme type has a moderate influence on academic risk. The one-hot encoded features `prog_Computer Science` (4th, 0.0778) and `prog_Data Science` (5th, 0.0775) ranked in the top 5, suggesting that students in these specific programmes have a slightly different baseline risk profile compared to IT or Software Engineering students (which ranked 8th and 13th). However, their importance scores are still lower than academic behavioural features, meaning programme alone is not enough to predict risk.

3. How does study hours per week relate to being at risk? Surprisingly, `study_hours_week` (6th, 0.0646) was a relatively moderate-to-weak predictor of academic risk. While we might assume that students who study less are more at risk, the model found that *attendance* and *assignment completion* are far more predictive. This suggests that merely spending hours studying is less important than actively attending classes and submitting coursework, highlighting the importance of structured academic engagement over independent study time.

Saving The Final Model For Application

```
import joblib

# Save the final XGBoost model and the scaler for deployment
joblib.dump(xgb_model, 'model.pkl')
joblib.dump(scaler, 'scaler.pkl')
print("Model and Scaler ready for deployment")
```

```
Model and Scaler ready for deployment
```

References

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32.

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

Çorbacıoğlu, Ş. K., & Aksel, G. (2023). Receiver operating characteristic curve analysis in diagnostic accuracy studies: A guide to interpreting the area under the curve value. *Turkish Journal of Emergency Medicine*, *23*(4), 195–198. https://doi.org/10.4103/tjem.tjem_182_23

Developers, S. (2024a). *Model selection and evaluation: Cross-validation*. https://scikit-learn.org/stable/modules/cross_validation.html

Developers, S. (2024b). *Preprocessing data: Encoding categorical features*. https://scikit-learn.org/stable/modules/preprocessing.html

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. https://hastie.su.domains/Papers/ESLII.pdf

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95.

McKinney, W. (2010). Data structures for statistical computing in python. *Proceedings of the 9th Python in Science Conference*, *445*, 51–56.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, *6*(60), 3021.

Zainol, Z. (2026). *Student performance classification dataset*.